

Guía a Vercel y Supabase

Pawlink

Guía práctica — Vercel y Supabase

Todo lo aprendido durante la configuración de Pawlink. Teoría y práctica mezcladas.

Supabase

¿Qué es?

Backend as a Service construido sobre PostgreSQL. Te da base de datos, autenticación, storage, y funciones serverless en un solo lugar, sin administrar servidores.

Extensiones (como librerías de PostgreSQL)

Se habilitan con `CREATE EXTENSION IF NOT EXISTS nombre`. Al igual que `npm install`, instalan funcionalidad extra en la base de datos.

```
CREATE EXTENSION IF NOT EXISTS postgis; -- coordenadas
geográficas
CREATE EXTENSION IF NOT EXISTS pgvector; -- embeddings de IA
(RAG)
```

Cuando habilitas PostGIS, Supabase instala automáticamente cientos de funciones internas (`geometry_*`, `st_*`) que puedes ver en **Database** →

Functions en el dashboard. No las creaste tú — vienen incluidas con la extensión.

PostgreSQL Functions vs Edge Functions

Son dos cosas completamente distintas con nombres parecidos:

	PostgreSQL Functions	Edge Functions
Dónde corren	Dentro del motor de PostgreSQL	Servidores Deno fuera de la DB
Lenguaje	SQL / PL/pgSQL	TypeScript (Deno)
Para qué	Lógica dentro de la DB, queries complejas	HTTP endpoints, llamadas a APIs externas
Cómo se llaman	<code>supabase.rpc('nombre')</code>	HTTP request a una URL
Ejemplos en Pawlink	<code>get_users_near_report()</code> , <code>get_reports_near_point()</code>	<code>social-post</code> , <code>geo-alert</code> , <code>adoption-</code> <code>confirmation</code>

Las que ves en **Database → Functions** en el dashboard son PostgreSQL Functions. Las Edge Functions viven en **Edge Functions** en el sidebar izquierdo.

PostgreSQL Functions en Pawlink

Creadas en `docs/schema.sql` y ejecutadas en Supabase:

`get_users_near_report(report_id, radius_m)` - Usa PostGIS (`ST_DWithin`) para encontrar usuarios registrados dentro de un radio - La llama el Edge Function `geo-alert` para saber a quién alertar

`get_reports_near_point(lat, lng, radius_m, ...)` - Busca reportes de mascotas perdidas/encontradas cerca de unas coordenadas - La llama `GET /api/lost-found` cuando el usuario pasa `lat` y `lng`

Se llaman desde el código así:

```
const { data } = await supabase.rpc('get_users_near_report', {
  report_id: report.id,
  radius_m: 2000,
})
```

Edge Functions

Código TypeScript que corre en Deno (no Node.js). Diferencias importantes:

- Los imports usan `jsr:` en lugar de `npm:` — por ejemplo `import { createClient } from 'jsr:@supabase/supabase-js@2'`
- Tienen un límite de 150 segundos (vs 10s de Vercel Functions)
- Se usan para workflows async disparados por la base de datos

Las tres Edge Functions de Pawlink:

Edge Function	Trigger	Qué hace
<code>social-post</code>	INSERT en <code>animals</code>	Llama Groq → genera post en español → guarda en <code>animals.social_post</code>
<code>geo-alert</code>	INSERT en <code>lost_found_reports</code>	Busca usuarios cercanos con PostGIS → manda emails via Resend
<code>adoption-confirmation</code>	UPDATE en <code>adoption_requests</code>	Manda email de confirmación a la familia via Resend

Edge Function	Trigger	Qué hace
	donde status → approved	

El código vive en `supabase/functions/<nombre>/index.ts` en el repo, pero **no se sincroniza automáticamente**. Después de cada cambio hay que hacer deploy manual:

```
supabase functions deploy social-post
supabase functions deploy geo-alert
supabase functions deploy adoption-confirmation
```

Vercel sí auto-despliega en cada push a main. Supabase Edge Functions no.

Database Webhooks

Son la forma en que Supabase dispara los Edge Functions automáticamente. Se configuran en **Database → Database Webhooks** en el dashboard.

Cada webhook escucha un evento en una tabla y hace un HTTP POST al Edge Function correspondiente:

```
INSERT en animals          → POST a /functions/v1/social-post
INSERT en lost_found_reports → POST a /functions/v1/geo-alert
UPDATE en adoption_requests → POST a /functions/v1/adoption-confirmation
```

El payload que recibe el Edge Function tiene esta forma:

```
{
  "record": { ...fila nueva o actualizada... },
```

```
"old_record": { ...fila anterior (solo en UPDATE)... }  
}
```

Los webhooks se configuran una sola vez en el dashboard. No viven en el repo.

Secrets en Edge Functions

Las variables de entorno para Edge Functions no son las mismas que las de Vercel. Se configuran por separado con el CLI de Supabase:

```
supabase secrets set GROQ_API_KEY=tu_key  
supabase secrets set RESEND_API_KEY=tu_key
```

Las variables de Supabase (`SUPABASE_URL` , `SUPABASE_SERVICE_ROLE_KEY`) están disponibles automáticamente dentro de cualquier Edge Function sin configurarlas.

Cientes de Supabase

Hay dos clientes con permisos diferentes:

Cliente público (frontend):

```
// Respeta RLS – el usuario solo ve lo que le corresponde  
import { createClient } from '@supabase/supabase-js'  
createClient(NEXT_PUBLIC_SUPABASE_URL,  
NEXT_PUBLIC_SUPABASE_ANON_KEY)
```

Cliente de servidor (backend / Edge Functions):

```
// Bypassa RLS – acceso total, usar con cuidado  
createClient(SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY)
```

Nunca exponer `SUPABASE_SERVICE_ROLE_KEY` en el frontend.

Storage

Dos buckets configurados en Pawlink:

Bucket	Visibilidad	Qué guarda
<code>pets</code>	Público	Fotos de animales
<code>documents</code>	Privado (solo shelter)	Documentos del refugio para RAG

Arquitectura de automatización

```
Usuario crea reporte de mascota perdida
→ POST /api/lost-found (Vercel Function)
→ INSERT en lost_found_reports (Supabase DB)
→ Database Webhook se dispara
→ geo-alert Edge Function recibe el payload
→ llama get_users_near_report() en PostgreSQL
→ por cada usuario cercano → POST a Resend API → email enviado
```

Vercel

¿Qué es?

Plataforma de hosting para aplicaciones frontend con soporte nativo para Next.js. Todo lo que está en `/api/*` se convierte automáticamente en serverless functions.

Auto-deploy

Cada push a `main` dispara un deploy automático en Vercel. No hay que hacer nada manual. El repo está conectado a Vercel desde la primera configuración.

Variables de entorno

Se configuran en **Project Settings → Environment Variables** en el dashboard de Vercel. Las variables con prefijo `NEXT_PUBLIC_` son accesibles desde el frontend. Las demás solo desde el servidor.

Variables necesarias para Pawlink:

```
NEXT_PUBLIC_SUPABASE_URL
NEXT_PUBLIC_SUPABASE_ANON_KEY
SUPABASE_SERVICE_ROLE_KEY
GROQ_API_KEY
AWS_ACCESS_KEY_ID
AWS_SECRET_ACCESS_KEY
AWS_REGION
RESEND_API_KEY
```

Límite de tiempo

Las Vercel Functions tienen un límite de **10 segundos** de ejecución. Para tareas que toman más tiempo (generación de posts, envío de emails en batch) se usan Supabase Edge Functions (150s).

tsconfig.json y carpetas Deno

Next.js intenta compilar todos los archivos `.ts` del proyecto, incluyendo los de `supabase/functions/`. Esos archivos usan imports de Deno (`jsr:`) que Node.js no entiende, lo que rompe el build.

Solución: excluir la carpeta en `tsconfig.json`:

```
{
  "exclude": ["node_modules", "supabase/functions"]
}
```

Git y ramas en este proyecto

Branch strategy

```
main (producción, auto-deploy a Vercel)
  ↑ PR
feat/nombre-descriptivo (desarrollo)
```

Problema común: branch desactualizado

Cuando se mergea a `main` desde GitHub, el clon local del repo no se entera. Si se crea un branch nuevo desde ese estado, arrastra commits que ya estaban en `main`.

Solución antes de pushear:

```
git fetch origin main
git rebase origin/main
git push origin nombre-branch --force
```

El `--force` es necesario después de un rebase porque reescribe el historial del branch.

Squash al mergear

Al hacer PR en GitHub, usar **Squash and merge** para que todos los commits del branch queden como uno solo en `main`. Mantiene el historial limpio.

Resend

Servicio de emails transaccionales. Reemplazó a N8N en Pawlink.

- **Free tier:** 3,000 emails/mes
- **Sin expiración de trial** — a diferencia de N8N Cloud (14 días)

- Se llama directamente desde los Edge Functions via HTTP a `https://api.resend.com/emails`

En Pawlink se usan dos remitentes: - `adopciones@pawlink.mx` — confirmaciones de adopción - `alertas@pawlink.mx` — geo-alertas de mascotas perdidas

Por qué usar Supabase Edge Functions y no Vercel Functions para las automatizaciones

Esta es una pregunta válida — ambas son serverless y ambas corren TypeScript. La diferencia está en **quién dispara la función y para qué**.

Vercel Functions

- Se disparan por **requests HTTP** — alguien llama a `/api/algo`
- Límite de **10 segundos**
- Diseñadas para responder al usuario en tiempo real (queries, matchmaking, vision)
- No tienen acceso directo a eventos de la base de datos

Supabase Edge Functions

- Se disparan por **Database Webhooks** — un evento en la base de datos (INSERT, UPDATE)
- Límite de **150 segundos**
- Diseñadas para workflows async que nadie está esperando
- Corren en la misma infraestructura que Supabase — acceso directo a la DB sin latencia extra

El problema concreto

Si usaras una Vercel Function para el geo-alert:

```
INSERT en lost_found_reports
→ Supabase Webhook → POST a Vercel Function /api/geo-alert
```

- Vercel Function llama a Supabase (query PostGIS)
- Vercel Function llama a Resend (emails)
- Tiene 10 segundos para hacer todo esto

Si hay 50 usuarios cercanos, mandar 50 emails en 10 segundos es arriesgado. Vercel cortaría la función a mitad del proceso y los emails se enviarían a medias o no se enviarían.

Con Supabase Edge Function tienes 150 segundos y corres más cerca de la base de datos, lo que reduce la latencia de las queries PostGIS.

Resumen de cuándo usar cada una

Situación	Usar
El usuario hace una acción y espera una respuesta	Vercel Function
Un evento en la DB dispara lógica async	Supabase Edge Function
Necesitas más de 10 segundos	Supabase Edge Function
Llamas a APIs externas en respuesta a un evento de DB	Supabase Edge Function
Query rápida o matching con IA	Vercel Function

En Pawlink la división es clara: - `/api/matching` , `/api/vision` , `/api/rag`
→ Vercel (respuesta en tiempo real al usuario) - `social-post` , `geo-alert` , `adoption-confirmation` → Supabase Edge (disparados por la DB, nadie espera la respuesta)

Por qué se reemplazó N8N

N8N Cloud tiene trial de 14 días. El Demo Day de Pawlink es 39 días después de la configuración inicial — el trial expiraría antes del demo.

La alternativa con Supabase Edge Functions + Resend es: - Gratuita sin límite de tiempo - El código vive en el repo (versionado) - Sin dependencia de un servicio externo de automatización - Más simple: el Edge Function hace todo en un lugar (query PostGIS + enviar emails) sin necesitar un intermediario HTTP